

Lab 07: Exploratory Data Analysis (EDA) with Unsupervised Learning

CSC 2621 Introduction to Data Science

Learning Outcomes

1. Extract features for non-tabular data
2. Apply dimensionality reduction methods to visualize high dimensional data
3. Compare and determine appropriateness of different data structures for a given situation
4. Apply clustering algorithms to unlabeled data
5. Characterize similarities and differences between clusters
6. Identify important features using statistical tests

Overview

In the first half of the class, we've been focusing on exploratory data analysis (EDA). You've primarily been using data sets with only a handful of variables. You could analyze each variable with visualizations and statistics to find relationships. High dimensional data sets, however, have too many variables for you to analyze each variable individually. We need to turn to more sophisticated techniques such dimensionality reduction and clustering.

In this lab, you are going to analyze 63,542 emails. You will convert the raw text into a feature matrix using a "bag of words" model. Each column of the feature matrix corresponds to one word, each row corresponds to one email, and the entry stores the number of times that word was found in that email. You will perform dimensionality reduction using the Truncated SVD method, cluster the emails, and compare the "inherent" structure to the given class labels.

Instructions

Part I: Load the Data

- a. Extract the provided email_json.zip file. It should create a directory called "email_json". Each email is stored as a separate JSON document with a name in the format "message_XXXXX.json".
- b. Write some code to find and load all of the JSON documents. You should have a list of dicts when done. (Hint: Review the built-in Python glob and json libraries.)
- c. Convert the list of dicts into a Pandas DataFrame. The DataFrame should have 5 columns and 63,452 rows. What are the column names and their types? (Hint: use the DataFrame.from_records() or DataFrame.from_dict() functions.)

Part II: Extract Features

By themselves, strings of the message bodies are not amenable to analysis. We need to convert them to a feature matrix.

a. Use Scikit Learn's [CountVectorizer](#) class with the `binary=True` flag to create a feature matrix from the message bodies and `min_df=10` to exclude any words that do not appear in at least 10 emails. The "bodies" column of the DataFrame can be used as a list of strings and passed directly into the `fit_transform()` method of the CountVectorizer.

b. How many rows and columns does the feature matrix have? How many nonzero entries are in the matrix?

Part III: Dimensionality Reduction

a. Use the `fit_transform()` function of Scikit Learn's [TruncatedSVD](#) class to transform the original feature matrix into a new feature matrix with 10 columns (variables or components). Scikit Learn's TruncatedSVD method is similar to its PCA method, but it works with sparse matrices.

b. Plot the explained variance ratios of the components. Which two components have the highest explained variance ratios? (Hint: use the `explained_variance_ratio_` property of the TruncatedSVD class.)

Part IV: Visualization

In part IV, we will plot the points along the two components you identified.

a. Create a scatter plot using the two components with the highest explained variance ratios. (Hint: `plt.scatter(proj_matrix[:, i], proj_matrix[:, j])`).

b. Create a second scatter plot using the same two components. This time, color the points based on the category column of the DataFrame. (All spam messages should be one color; all ham messages should be a second color.)

Part V: Cluster the Emails

To perform further analysis, we want to cluster the emails. By clustering the emails, each message will be assigned a cluster id (e.g., 0, 1, 2, etc.).

a. Review the [clustering algorithms](#) available in Scikit Learn. Choose a clustering algorithm that you think would be appropriate for this data set.

b. Cluster the samples using the two SVD components that you determined to have the highest explained ratio. The clustering algorithm should return a 1D Numpy array of cluster labels (e.g., 0, 1, 2, etc.) for each point.

c. Create a scatter plot along the 2 SVD components. Color the points according to their cluster labels.

d. Assess the cluster assignments. Visually, the data should form two distinct groups or clusters. The clustering algorithm should label the points so that all points in the same cluster have the same cluster id. (With two clusters, there should only be two cluster ids.) If the clustering does not look correct, try adjusting the parameters of the clustering algorithm you chose or choose a different algorithm.

e. Calculate a confusion matrix for the ham / spam labels versus the cluster labels.

Part VI: Calculating Document Frequencies of Words

From part V, we now have clusters, but we don't know why certain those specific emails were clustered together.

a. Create a separate matrix for each cluster containing the rows for the points in that cluster.

b. Convert the matrices to CSC format. We will be accessing the data column-wise. Column indexing is significantly faster for the CSC format than the CSR format. (Hint: Review the `scipy.sparse` module.)

c. Calculate the document frequency of each word for each cluster. The document frequency is the number of documents that contain each word. Since the feature matrix is binary, we can simply sum along the columns. (Note: The resulting matrices should have the same shapes with one entry for each word in the vocabulary. If not, then you may have done the sum along the rows instead of the columns.)

Part VII: Find Enriched Words with Statistical Testing

We are going to use a Binomial test to determine if the number of occurrences of a given word in a given cluster is higher than what would be expected from the other cluster.

Here is the code for calculating the expected probability of a word based on cluster 1 and using it to test if the word is enriched in cluster 0:

```
cluster_1_expected_prob = doc_freq_cluster_1 / num_emails_cluster_1
pvalue = stats.binom_test(doc_freq_cluster_0, num_emails_cluster_0,
                          cluster_1_expected_prob, alternative="greater")
```

The null hypothesis that the relative document frequencies of the observed cluster are less or equal to those of the tested. The alternative hypothesis is that the document frequency is higher in cluster 0 than cluster 1.

a. Try testing if the words "works" and "love" are enriched in cluster 0.

For "works", you should get an expected probability of 0.109 for cluster 1 and observed probability of 0.041 for cluster 0. The Binomial test should return a p-value of 0.999,

indicating that the observed frequency for cluster 0 is NOT greater than the frequency for cluster 1.

For "love", you should get an expected probability of 0.004 for cluster 1 and observed probability of 0.035 for cluster 0. The Binomial test should return a p-value of 0.0, indicating that the observed frequency for cluster 0 is greater than the frequency for cluster 1.

b. Wrap your code for part (a) in a loop to find enriched words for cluster 0. Calculate the p-value for every word. If the p-value < 0.05, add a tuple of (pvalue, word, cluster 0 document frequency to a list. (Hint: Iterate over the vocabulary_ dict to get each word and its index.)

c. Filter out any words that contain non-alphabetic characters. (Hint: Use the isalpha() method of strings.)

d. Sort the words in ascending order by their p -values and print out the first 200 words.

e. Repeat with the clusters reversed so you can find words enriched in cluster 1.

Reflection Questions

1. The vectorizer returns a sparse matrix in compressed row format (CSR). Assume that the sparse matrix uses one 32-bit (4-byte) floating point number and one 32-bit (4-byte) integer for each nonzero entry and one 32-bit (4-byte) integer for each row. Calculate the memory usage for the sparse matrix.

2. Calculate the sparsity ratio (100 * number of nonzero entries divided by maximum possible entries).

3. Skim through the top 200 words for each cluster. Can you identify any patterns for either of the clusters?

4. Compare the ham/spam labels to the cluster labels using the confusion matrix you generated. Are spam messages in both clusters or a single cluster? Are all of the messages in the clusters with spam labeled as spam?

Submission Instructions

Save the Jupyter notebook as a PDF and upload that file through Canvas.

Rubric

Followed submission instructions	5%
Formatting: Report is polished and clean. No unnecessary code. Section headers are used. Plots are described and interpreted using text. Plots have axis labels. The report contains an introduction and conclusion.	5%
Part I: Load the Data	
Found and loaded all JSON documents	10%
Converted JSON documents to a DataFrame	10%
Part II: Extract Features	
Extracted word counts to create feature matrix	10%
Part III: Dimensionality Reduction	
Extracted 10 components using SVD	10%
Plotted the explained variance ratios and choose the 2 correct components	5%
Part IV: Visualization	
Scatter plot (with labels)	5%
Part V: Clustering the Emails	10%
Part VI: Calculate the Document Frequency of the Words	10%
Part VII: Find Enriched Words with Statistical Testing	10%
Response Questions	10%